CORE JAVA SERIES — DAY 6

Functions, Recursion & Intro to OOP

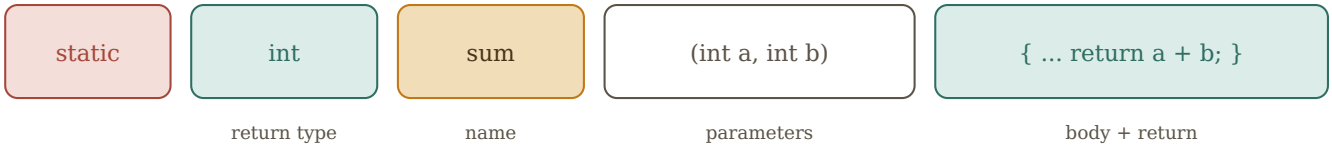
Reusable code with functions (types, overloading, scope, chaining, recursion), then the leap into Object-Oriented Programming: classes, objects, reference variables, and modeling real-world things in code.

Why Functions?

Suppose you need to add two numbers in several places in your program. Without functions you'd redeclare variables and rewrite the same logic every time — a lack of **code reusability**. A **function** is a named block of code that does one job; write it once, then **call** it whenever needed.

> **Math analogy:** just like f(x) takes an input, processes it, and gives an output, a Java function takes inputs, does some work, and (optionally) returns a result.

Anatomy of a Function



Functions live **outside** main (but inside the class). `static` is required for now — its reason becomes clear after OOP.

- **Return type** — the data type the function hands back (`void` if it returns nothing).
- **Parameters** — the variables the function expects as input (type + name each).
- **Arguments** — the actual values you pass when **calling** the function, e.g. `sum(4, 5)`. They get assigned to the parameters.
- **return** — sends the result back to the caller.

> **Parameter vs argument:** parameters are what the function *expects* (placeholders); arguments are what you *actually pass* (real values). Calling `sum(i, j)` copies i into a and j into b.

Four Kinds of Functions

Kind	Example
No input, no output	`static void greet() { println("Hello"); }`
Input, no output	`static void sayHello(String name)`
No input, output	`static int getNumber() { return 10; }`
Input and output	`static int multiply(int a, int b)`

- The number of parameters can be anything — zero, one, or many.
- For `void` functions, a bare `return;` is optional.
- You may **ignore** a returned value (just call the function without storing it), or use the call directly inside another expression: `System.out.println(getNumber());`

main Is Also a Function

`public static void main(String[] args)` is simply a function: name `main`, return type `void`, one parameter (a String array). It is the **entry point** — the JVM looks for it and starts executing line by line from there.

> **Consequence:** a function you define but never call (directly or indirectly) from main will never run. Everything you want executed must be reachable from main.

Flow of a call: control jumps into the called function, executes it, then **returns to exactly where it was called** and carries on with the next line.

Function Overloading

Java lets multiple functions share the **same name** as long as the compiler can tell them apart by their parameters. They can differ by:

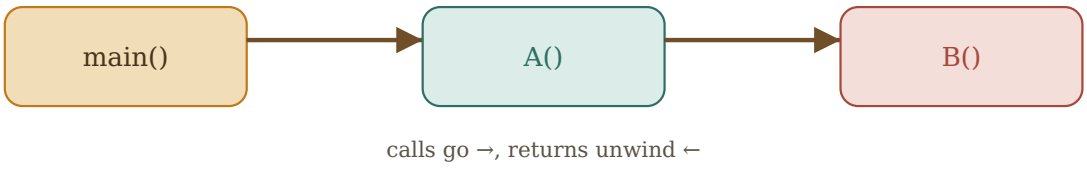
- **Number of parameters** — `sum(int a, int b)` vs `sum(int a, int b, int c)`
- **Type of parameters** — `sum(int, int)` vs `sum(double, double)`
- **Order of parameters** — `greet(String name, int age)` vs `greet(int age, String name)`

At the call site, the arguments you pass decide which version runs.

> **Return type alone is NOT enough.** Two functions `void fun()` and `int fun()` cause a "*duplicate method*" compile error — because you're allowed to call a function and ignore its result, the compiler can't tell which one you meant. Change the parameters (or the name) instead.

Chaining of Functions

One function can call another, which calls another, and so on: `main → A → B`. Control goes **down** the chain as far as it goes, then unwinds **back up** in reverse order, each function finishing whatever code is left below its call.



calls go →, returns unwind ←

If B prints "Hello", A prints "Hi" after calling B, and main prints "Bye" after calling A, the output order is Hello → Hi → Bye. (The mechanism behind this is the **stack memory**, covered later.)

Scope of a Variable

- **Local scope** — a variable declared inside a pair of curly braces (a function, `if`, loop, etc.) exists only inside that block. When the block ends, the variable is destroyed. A variable `x` declared in `main` is **not visible** inside another function — you must pass it as an argument, or the other function needs its own.
- **Global scope** — a variable declared at class level (outside every function) is visible to all functions in the class. For now it must be marked `static`: `static String name = "Aditya";`

> **Error hint:** "*x cannot be resolved to a variable*" almost always means you're using a variable outside its scope.

Active Recall — Functions

Q1. Difference between a parameter and an argument?

A. Parameters are the placeholders in the function definition; arguments are the actual values passed at the call.

Q2. What does void mean?

A. The function returns nothing.

Q3. Why does declaring void fun() and int fun() together fail?

A. Overloading can't rely on return type — since the returned value can be ignored, the compiler can't tell which one a call refers to.

Q4. How can main's variable be used in another function?

A. Pass it as an argument (received as a parameter), or make it a class-level (global) static variable.

Q5. What happens to a function that is defined but never called?

A. It never executes — the JVM only runs main and whatever main calls.

Recursion

Recursion = a function that calls **itself**. Without a stopping point it loops forever, so every recursive function needs:

- **Base case** — the condition where it stops calling itself and simply returns.
- **Recursive call** — the self-call, with a smaller/changed argument moving toward the base case.

Example: Print 1 to n

```
static void printNum(int n) { if (n == 0) return; printNum(n - 1); System.out.println(n); }
```

Trace for n = 5: `printNum(5)` calls `printNum(4)` → … → `printNum(0)` hits the base case and returns. Then the calls **unwind**: the code after each recursive call runs in reverse — print 1, then 2, 3, 4, 5. Output: **1 2 3 4 5**.



calls go right → base case → each level then prints n while returning left

Think of stairs: walk down to the base, then climb back up doing the remaining work.

No base case = infinite recursion. printNum would keep going to 0, −1, −2, … and never stop.

Example: Fibonacci

Each number is the sum of the previous two (1, 1, 2, 3, 5, 8 …). Recursively: `fib(n) = fib(n−1) + fib(n−2)`, with base case `n == 0 || n == 1 → return 1`. You only describe **one layer** — "get the previous two and add them" — and recursion handles the rest.

Unlike printNum (a single chain), Fibonacci makes **two** recursive calls per level, so the calls form a **tree**: leaves are base cases, and values are added together as the tree collapses upward.

When to use recursion: for simple loops an iterative `for` is easier, but recursion shines on problems that are naturally self-similar or too tangled for plain loops — common in Data Structures & Algorithms. Practice on easy ones first (print 1→n, n→1, reverse).

Active Recall — Recursion

Q1. What is recursion?

A. A function that calls itself, each time with a smaller/changed input.

Q2. What is a base case and why is it essential?

A. The condition where the function stops recursing and returns; without it the calls never end.

Q3. In printNum, why does the output come out 1→n rather than n→1?

A. The print statement is placed after the recursive call, so it executes while unwinding — the deepest call (smallest n) prints first.

Q4. Why do Fibonacci's recursive calls form a tree, not a chain?

A. Each call makes two recursive calls (n−1 and n−2), so calls branch at every level.

Enter Object-Oriented Programming: The Problem

Suppose we store one student's data with plain variables: `String name; int age; int rollNumber; String college;`. Problems appear immediately:

- A lot of **independent variables** to represent one student — nothing in the code says they belong together.
- To pass a student to a function you must pass **all four variables**; forget one and it breaks.
- A second student needs a whole **new set** (`name2, age2, …`); a third needs another, and so on.
- **No authority over data** — nothing owns these variables; anyone can change them independently.

OOP is the answer. It is a whole **programming paradigm** (a way of thinking about and writing code); Java is an object-oriented language.

The Core Idea: Mimic the Real World

We write code to solve real-world problems, so OOP says: **model the real world in code**. Real-world things (living or not) are **objects**. A student is an object with:

Characteristics (data)	Behaviors (actions)
name, age, roll number, college	mark attendance, …

Instead of four loose variables, we build **one block** that holds all of a student's data together.

Model Only What Matters

A real human can be a student, a son/daughter, an employee — each role has different characteristics. We do **not** copy the whole person. We capture only the characteristics **relevant to the problem**. For a college portal, that's the student role (name, age, roll number, college); for an HR app, it would be employee data. Pick what your program actually needs.

Class vs Object

- **Class** — the abstract idea / **blueprint**: "what a student looks like" (which fields it has). No memory for actual data is allocated by just writing the class.
- **Object** — a concrete, real instance built from that blueprint: an actual student such as Aditya or Rohit, with real values.

> `class Student { String name; int age; int rollNumber; String college; }` — declared **outside** main (and outside the main class body's function). This is a **user-defined data type**; classes/objects belong to the **non-primitive** types.

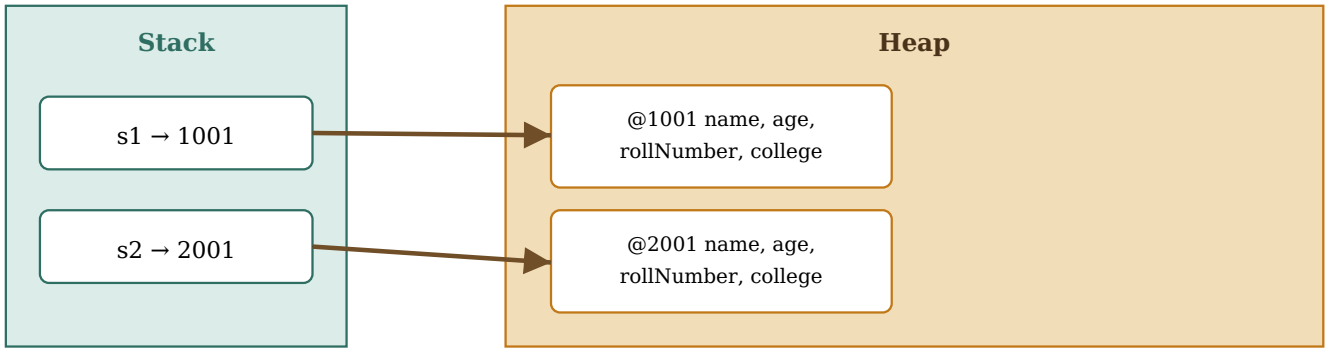
Creating an Object

```
Student s1 = new Student();
```

- `new Student()` — the `new` keyword allocates a block in **heap memory** big enough for a Student object (with its

own name, age, rollNumber, college).

- `Student s1` — a **reference variable** living in **stack memory**; it stores the object's starting address (it points to it).



Reference variables (s1, s2) sit in the stack; the actual objects sit in the heap.

> **Interview classic:** `s1` is **not** the object — it is only a reference variable holding the object's address. The object itself in the heap has **no name**; we reach it through references.

Accessing Fields: the Dot Operator

`s1.name = "Aditya"; s1.age = 28; s1.rollNumber = 1001;` — the **dot operator** follows the reference to the object, then picks a field. Read the same way: `System.out.println(s1.name);`. You can create as many objects as you like (`Student s2 = new Student();`) — each has its own separate data.

Compile Time vs Run Time

`new` works at **run time**, so it is **dynamic memory allocation** (in the heap). Simple variables like `int x = 4;` are handled at compile time — **static** allocation.

Behaviors = Functions Inside the Class

Real objects also *do* things. Take an `Animal`: characteristics (type, number of legs) plus behaviors (make sound, eat, run). Characteristics become **variables**; behaviors become **functions** written inside the class.

```
class Student { String name; int age; ... void markAttendance() { System.out.println("Attendance marked by " + name); } void print() { System.out.println(name + ", " + age + ", " + rollNumber + ", " + college); } }
```

- Call with the dot operator: `s1.markAttendance(); s2.print();`
- These functions need **no parameters** for the object's own data — they belong to the object and can already access its fields (`name` means *this object's* name).
- Instead of the old `print(String name, int age, …)`, you can either pass one whole `Student`, or better, let the student **print itself**.
- A behavior like `print()` doesn't exist for real students — but a **digital** student can have it. Objects don't need to mirror reality exactly; add whatever makes the code cleaner.

Non-Physical Objects Too

Anything complicated enough that a single variable can't represent it can be a class — even non-physical things: `BankAccount` (account number, IFSC, branch), `Location` (latitude, longitude), and so on.

Naming Conventions

Not compiler rules — just good practice that all Java code follows:

Item	Style	Example

Variables (primitive & reference)	camelCase	`firstName`, `rollNumber`
Functions	camelCase	`markAttendance`
Classes	Starts with a capital (PascalCase)	`Student`, `BankAccount`

Java Is (Almost) Fully Object-Oriented

Every Java program must live inside a class. Even the boilerplate is a class: `public class Demo { public static void main(String[] args) { ... } }` — a class named Demo containing the `main` function. Our own classes (like `Student`) are defined outside it, and their objects are created **inside main** (only then is any memory allocated). The details of `public`, `static`, and `String[] args` will make sense as OOP deepens.

Active Recall — OOP Intro

Q1. Name two problems with representing a student using independent variables.

A. Many loose variables with nothing grouping them; you must pass every variable to functions; every new student needs a fresh set; no single owner of the data.

Q2. Class vs object?

A. A class is the blueprint/abstract idea; an object is a real instance built from it with actual data, stored in the heap.

Q3. What does Student s1 = new Student(); create, and where?

A. new creates a Student object in the heap; s1 is a reference variable in the stack holding that object's address.

Q4. Is s1 the object?

A. No — s1 is just a reference variable; the object itself is unnamed and lives in the heap.

Q5. Why can markAttendance() use name without taking it as a parameter?

A. The function belongs to the object, so it has direct access to that object's own fields.

Q6. Why is new called dynamic memory allocation?

A. Because the heap memory is allocated at run time, not at compile time.

Q7. Should we copy every real-world characteristic into a class?

A. No — only the ones relevant to the problem the program solves.